

Performance profiles for the second-order radius rules on CUTEst

Abstract

Six radius update rules are run on the unconstrained CUTEst set in two arms, a first-order and a second-order one, and in two passes, so that the cost each arm had spent when it first met each of two criteria is measured rather than inferred. Every run is then scored after the fact, from the point it returned, by one criterion applied by the same code to every run and never by the solver's own status. 190 problems ran, 4 560 runs in all.

Of the three profiles, P1 stands as designed: on the second-order task the six rules separate by at most five percentage points in robustness and by nearly a factor of three in cost, with R-gRTR jointly the most robust and the cheapest and R-DFO last on both counts. P2 stands with one qualification and one figure withdrawn: the second-order arm is *more* robust than its first-order counterpart on the *first-order* task, for every rule, and spends fewer Hessian-vector products doing it — but the arms differ in three things at once, so this compares configurations and not second-order-ness. P3 measured nothing. Its first-order columns read 0 on every rule because the first-order arm never records $\lambda_{\min}$, so the criterion rejects those runs before consulting a single numerical fact about them.

What the archive does still answer changes how the rest should be read. On this problem set, at these tolerances, the point at which $\|g\| \leq \varepsilon_g$ first held *already* satisfied $\lambda_{\min} \geq -\varepsilon_H$ on 96.8–98.1% of problems, and the median cost of certifying the curvature beyond that point was exactly zero extra iterations. The second-order machinery here buys a certificate rather than a better point, at full price on every problem, to matter on about five — and where it matters it matters completely.

1 What is measured

Three performance profiles, in this order of importance.

- P1** Among the second-order radius rules, on the second-order task. Only the radius rule differs across the lines, so this profile needs no further argument.
- P2** Second-order against first-order, scored on the *first-order* task. This is the overhead of carrying the curvature machinery when a first-order point is all that was wanted.
- P3** Second-order against first-order, scored on the *second-order* task. This is the payoff.

P2 and P3 together are how the two families compare. Neither alone is an answer. A report quoting only P2 says the second-order arm is expensive, a report quoting only P3 says it is reliable, and each has described half of one trade.

1.1 The rule that makes the comparison valid

Every run is scored afterwards, from the point it returned, against one criterion applied by the same code to every run. No profile in this report is drawn on the solver's own status.

The reason is that the two families stop on different tests. A first-order radius rule stops when the gradient is small. A second-order rule keeps going until it also has curvature. Profiling

the two on their own stopping tests compares the cost of an easy task with the cost of a hard one, and the first-order rules win by construction.

Two tolerances are fixed once,

$$\varepsilon_g = 10^{-5}, \quad \varepsilon_H = 10^{-6}. \quad (1)$$

Both are the package's shared settings, ε_g the `tol` of `SOLVER_PARAMS` and ε_H the `tol_H` of `SECOND_ORDER_PARAMS`. The two criteria are

$$\text{first-order : } \|g\| \leq \varepsilon_g, \quad (2)$$

$$\text{second-order : } \|g\| \leq \varepsilon_g \text{ and } \lambda_{\min} \geq -\varepsilon_H. \quad (3)$$

Both are evaluated by `second_order_status` of the package, which is the same function the solver's own stopping test uses, called here on the returned point of every run whatever that run's status was.

2 Three things established before the runs

2.1 The subproblem solver

Part II asks the step for a fraction of the decrease available along the leftmost eigenvector. Assumption as: `Eigenpoint decrease` requires

$$m_k(0) - m_k(s_k) \geq \frac{1}{2} \kappa_{\text{fed}} |\lambda_k^-| \Delta_k^2 \quad \text{whenever } \lambda_k^- < 0, \quad (4)$$

and as: `max Cauchy/eigen decrease` asks for the better of that and the Cauchy decrease. Theorem `thm`: `Eigenpoint decrease` of Part II supplies (4) for the eigenpoint $s_k^E = \Delta_k u_k$ of `eqn`: `eigenpoint`, with u_k any direction satisfying `eqn`: `sufficient negative curvature direction`.

Three subproblem solvers of the package were examined against that condition.

ExactMS

Solves the trust-region subproblem exactly by the Moré–Sorensen characterisation, hard case included, so it dominates both the Cauchy point and the eigenpoint and satisfies (4) with $\kappa_{\text{fed}} = 1$. It needs a dense eigendecomposition and refuses above its `nmax`, which is 200.

EigenPoint(SteihaugCG())

Computes $\pm \Delta v_{\min}$ and returns whichever of that and the inner step decreases the model more, so it satisfies (4) with $\kappa_{\text{fed}} = \kappa_{\text{snc}}$, the quality of the eigenvector. That is 1 in the dense branch and Lanczos-limited above it.

SteihaugCG

Terminates on the first direction of negative curvature the conjugate-gradient recurrence reaches and runs to the boundary along it. The decrease is real and the direction is whichever the recurrence happened to find, so the guaranteed decrease carries no $|\lambda_k^-| \Delta_k^2$ term. It does not deliver the eigenpoint decrease.

The second-order arm therefore uses `ExactMS` at $n \leq 200$ and `EigenPoint(SteihaugCG())` above. The switch is on the problem, not on the column, so every line of a profile sees the same subproblem solver on a given problem and the per-problem normalisation is unaffected. The first-order arm uses the package's default, `SteihaugCG`. Every caption names the pair.

2.2 The criticality measure

Part II defines the second-order criticality measure at `eqn:tau definition`,

$$\tau_k := \max\{\|g_k\|, -\lambda_k^-\} \geq 0, \quad (5)$$

with λ_k^- the leftmost eigenvalue of the *model* Hessian H_k . Part II states that τ_k is a property of the pair (x_k, H_k) and not of x_k alone, for that reason.

The package computes τ in `tau_criticality` as $\max\{\|g\|, -\min(\lambda_{\min}, 0)\}$. The inner clamp is a no-op inside the maximum, since $\|g\| \geq 0$. The two forms were compared on a grid of twenty pairs $(\|g\|, \lambda)$ covering $\lambda < 0$, $\lambda = 0$ and $\lambda > 0$, and the worst absolute difference is 0. So the package implements (5) exactly and no choice arises.

The open note at `app: measure` of the numerical study says that τ_k is the leftmost model Hessian eigenvalue in Part I and the whole second-order measure in Part II, and that Part III must pick one. **The premise is inaccurate and there is nothing to pick.** Part I defines no second-order criticality measure. Its criticality-anchored section carries a generic measure χ_k with $\chi_k = \|g_k\|$ as its stated example, at `eqn:criticality step`, and its concluding section says that second-order stationarity is outside the scope of its framework. The string `eigen` does not occur anywhere in the Part I source, and its four occurrences of `lambda_` are the rates λ_1 and λ_2 of the adaptive update. The note should be resolved by recording that τ_k is (5) of Part II, which is what the package already implements.

2.3 Per-iteration costs, and the route taken

Post hoc scoring needs the cost a run had spent when it *first* met the criterion, not its total at the end. The package's trace does not carry that. The trajectories it records are the radius, the gradient norm, the objective, the ratio, the step norm, the activity and acceptance flags, τ , the curvature and a further nine diagnostics. Not one of them is an evaluation count. The benchmark record adds three totals, read off the model after the solve, and no per-iteration counts either.

The fallback route was taken. Each configuration is run twice, with the tightened criterion as its *actual* stopping rule.

Pass g

halts at the first iterate with $\|g_k\| \leq \varepsilon_g$. Its cost is what P2 scores.

Pass h

halts at the first iterate that also has $\lambda_{\min} \geq -\varepsilon_H$. Its cost is what P1 and P3 score.

This is more runs and it is exact. Both passes keep their arm's own `tol_H`, which in this package controls the curvature estimate as well as the stopping test. Switching it off in pass g would also switch off the cost that pass exists to measure, so pass g halts through a callback instead.

One arithmetic consequence of the package's design bears on Section 6. With `ExactHessian` at $n \leq 200$ the curvature estimate goes through `dense_hessian`, hence through `NLPModels.hess`, and is paid in *Hessian evaluations*. Above 200 it goes through Lanczos on `hessian_op` and is paid in Hessian-vector products. A profile on Hessian-vector products alone would therefore show the second-order arm as free on the small problems, which is the opposite of true. Both axes are reported.

3 The configurations

Six radius rules, in two groups of three.

Table 1: The roster. The second column is `is_criticality_anchored` and the third `asymptotic_regime`, both read from the package rather than assumed. ω_k is the measure the radius is anchored to in the second-order arm.

rule	reads a measure	regime	ω_k in the second-order arm
R-delta	no	<code>bounded_below</code>	not read
R-step	no	<code>step_summable</code>	not read
R-RTR	no	<code>step_summable</code>	not read
R-DFO	yes	<code>vanishing</code>	τ_k
R-grad	yes	<code>vanishing</code>	τ_k
R-gRTR	yes	<code>vanishing</code>	τ_k

The distinction in Table 1 governs how every caption in this report must be read. R-DFO, R-grad and R-gRTR read a criticality measure, so replacing $\|g_k\|$ by τ_k changes the radius rule itself. R-delta, R-step and R-RTR do not read one, so for them the second-order arm is *the same radius rule under a different stopping test and a different subproblem solver*. Both groups belong in P1, and a caption that does not say which is which claims a distinction the figure does not have.

The two arms differ in exactly three things.

Table 2: The two arms. Everything not listed is identical, including the model Hessian, the acceptance and scaling thresholds, the contraction and expansion factors, the initial radius, the iteration cap and the two tolerances of (1).

	first-order arm	second-order arm
second-order stopping test	off	on, $\varepsilon_H = 10^{-6}$
ω_k , where read	$\ g_k\ $	τ_k
subproblem solver	<code>SteihaugCG</code>	<code>ExactMS</code> ($n \leq 200$), <code>EigenPoint(SteihaugCG())</code> ($n > 200$)

The shared settings are those of `SOLVER_PARAMS`, unchanged: $\eta = \eta_1 = 0.1$, $\eta_2 = 0.9$, $\gamma_1 = 0.25$, $\gamma_2 = 0.5$, $\gamma_3 = 2$, $\Delta_0 = 1$, $\Delta_{\min} = 0$, and at most 10^4 iterations. R-DFO runs at $\zeta = 100$ and R-grad and R-gRTR at $\mu_0 = 1$ with no cap. The two retrospective rules run at their own $\tilde{\eta}_1 = 0.05$ and $\tilde{\eta}_2 = 0.9$. Per-rule tuning would measure tuning effort rather than algorithmic merit, which is why the settings are shared.

The model Hessian is `ExactHessian` in both arms. That choice is what makes the criterion (3) a statement about f . It was checked rather than assumed. On the problems tested, the model's dense Hessian agrees with `NLPModels.hess` entry by entry, and the package's $\lambda_{\min}$ agrees with a direct dense eigendecomposition, both to 0 in floating point.

3.1 LBFGSModel gets no line

`LBFGSModel` enforces $B \succ 0$, so $\lambda_{\min} > 0$ at every iteration, so $\tau_k = \|g_k\|$ identically and the second-order variant of a rule *is* its first-order variant. It therefore has no line of its own in P1.

This was verified on iterate sequences and not asserted. For each of the three criticality-anchored rules, on two problems, the wrapped and unwrapped rules were run over `LBFGSModel` and their iteration counts, gradient trajectories and radius trajectories compared. All six pairs are identical, entry by entry. There is no difference to report as a finding.

4 The problem set

CUTEst, unconstrained, with all variables free, $2 \leq n \leq 1000$ and no constraints. CUTEst version 1.4.0. The query returns **196** problems, of which **185** have $n \leq 200$ and **11** have $n > 200$. The realised list of names and dimensions is written beside the results, in `exp17_problems.txt`.

The fallback was disabled deliberately. `default_problems()` of the benchmark returns the analytic set when the CUTEst query comes back empty, so a run in which CUTEst failed to load produces a table that reads as a CUTEst benchmark and is not one. The experiment checks `HAS_CUTEST`, checks that the query returned something, and raises rather than falling back.

P3 runs on the 185 problems with $n \leq 200$ and no others. Above that dimension the package estimates $\lambda_{\min}$ by Lanczos, a Ritz value satisfies $\lambda_{\text{Ritz}} \geq \lambda_{\min}$, and an under-resolved run therefore *understates* the negative curvature and makes the second-order test optimistic. Scoring P3 on such an estimate would rank the runs by the machinery the profile exists to compare. On the 185 problems the estimate is a dense symmetric eigendecomposition of $\nabla^2 f(x)$ and is exact.

5 The realised campaign

The query of Section 4 returned 196 names. 190 of them opened; 6 raised at construction and never reached the solver. Those 6 are `ARGLINA`, `ARGLINC`, `BROWNAL`, `CERI651BLS`, `CERI651DLS`, `CERI651ELS`. Section 11 takes up what that costs the shipped tables; here it is enough to record that every count in this report is denominated by the 190 problems that produced data, of which 179 have $n \leq 200$ and 11 have $n > 200$.

Six rules, two arms, two passes gives 24 runs per problem and 4560 in all. Figure 1 is the run matrix for one rule: the four cells, what halts each, and which profile reads which.

	<i>pass g</i> halts at $\ g_k\ \leq \varepsilon_g$	<i>pass h</i> halts on the arm's own test
first-order arm SteihaugCG <code>tol.H = -1</code>	Reaches a first-order point. No curvature is computed at any iterate: H evaluations = 0. Read by P2.	Reaches a first-order point. $\lambda_{\min}$ is never recorded, so the archived value is NaN. Read by P3 , where it scores 0 before any number is consulted.
second-order arm ExactMS <code>tol.H = ε_H</code>	Reaches the same first-order point, having paid for $\lambda_{\min}$ at every iter- ation. Read by P2.	Reaches a point with $\ g\ \leq \varepsilon_g$ and $\lambda_{\min} \geq -\varepsilon_H$. Read by P1 and P3.

Figure 1: The run matrix for one rule. Three cells carry information. The dashed cell is where P3 is lost: with `tol.H = -1`, an unwrapped rule and a subsolver that requests no eigenvector, the solver sets `want_curv = false`, `attach_trace!` omits `:lambda_min_trajectory`, and the criterion (3) has no $\lambda_{\min}$ to test.

Table 3: Status composition of the 4560 archived runs, both passes pooled. `user` is the pass-g callback of Section 2.3 firing, so it counts pass-g runs that reached a first-order point. The last line is the subject of Section 9.

	first-order arm	second-order arm
<code>first_order</code>	931	0
<code>second_order</code>	0	1 006
<code>user</code> (pass g callback)	930	1 008
<code>max_iter</code>	72	30
<code>max_time</code>	139	128
<code>stalled</code>	208	104
<code>exception</code>	0	4
runs	2 280	2 280
runs with a finite $\lambda_{\min}$	0	2 276

Two settings of the realised run differ from the defaults quoted in Section 3 and should be recorded. The per-run time limit was 60 s, set through `TRR.SO_MAXTIME`, not the 120 s of `SOLVER.PARAMS`; the archive’s `extra` block carries the 60 and the `params` block it writes beside it still carries the 120, so the two disagree and the runs used the smaller. It binds: 70 first-order and 64 second-order runs of pass g ended on it. And the campaign consumed about five and three quarter hours of wall time.

6 The two cost axes

Section 2.3 predicted that the curvature estimate would be paid in Hessian evaluations below $n = 200$ and in Hessian-vector products above it, and that a profile on products alone would show the second-order arm as free. The run bears this out in the sharper form: across all 2 280 first-order runs the Hessian-evaluation count is *identically zero*. That arm never forms a Hessian on any problem at any dimension, because nothing in its configuration asks for curvature.

Two consequences follow, and they pull in opposite directions.

1. The second-order arm’s Hessian evaluations are pure overhead with no counterpart in the other arm, so on that axis the comparison is 0 against a positive number and there is no ratio to profile. This is why `exp17_P2_h_evals.pdf` is withdrawn in Section 11.
2. On Hessian-vector products the sign reverses. `SteihaugCG` iterates where `ExactMS` factorises, so it is the *first-order* arm that spends products, by a factor of 1.96 to 5.74. Reporting either axis alone would misstate the trade in one direction or the other.

7 P1: the six rules on the second-order task

Table 4: P1. Second-order arm, pass h, scored by (3) at the returned point. Costs are totals over all 190 problems, met or not: no filtering on success and no mean over survivors. $\rho(\infty)$ is the robustness of the corresponding profile line. Subproblem solver `ExactMS` at $n \leq 200$, `EigenPoint(SteihaugCG())` above.

rule	met	$\rho(\infty)$	f	g	Hv	H	wall (s)
R-delta	166	0.874	53 994	48 958	189 068	96 585	800
R-step	167	0.879	67 150	63 810	245 382	115 716	855
R-RTR	167	0.879	59 972	53 079	220 668	104 164	869
R-DFO	164	0.863	118 206	106 469	260 899	216 522	815
R-grad	171	0.900	52 792	44 539	196 231	82 731	795
R-gRTR	171	0.900	40 357	33 933	239 321	60 598	824

Robustness spans 164 to 171 of 196, a range of five percentage points. Cost spans nearly a factor of three. The separation between these rules is therefore in what they spend, not in what they reach, and three readings follow from Table 4.

R-gRTR is first on both counts.

It meets the criterion on the same 171 problems as **R-grad** while spending 23.6% fewer objective evaluations and 26.8% fewer Hessian evaluations, and 72.0% fewer Hessian evaluations than **R-DFO**.

R-DFO is last on both counts.

Fewest problems met, 164, and the largest bill: $2.93\times$ the objective evaluations of **R-gRTR** and $3.57\times$ its Hessian evaluations. Under $\omega_k = \tau_k$ the $\Delta_k > \zeta \|g_k\|$ test of **R-DFO** is not paying for itself here.

The two retrospective rules buy their products.

R-RTR and **R-gRTR** require $\tilde{\rho}$, hence one further Hessian-vector product at the trial point on every accepted step. With `ExactMS` solving the subproblem that extra product is most of the product count, which is why both sit near twice the field on the Hv axis of Table 4 while their other columns are competitive or better. It is an accounting fact about the retrospective ratio and not a defect of either rule.

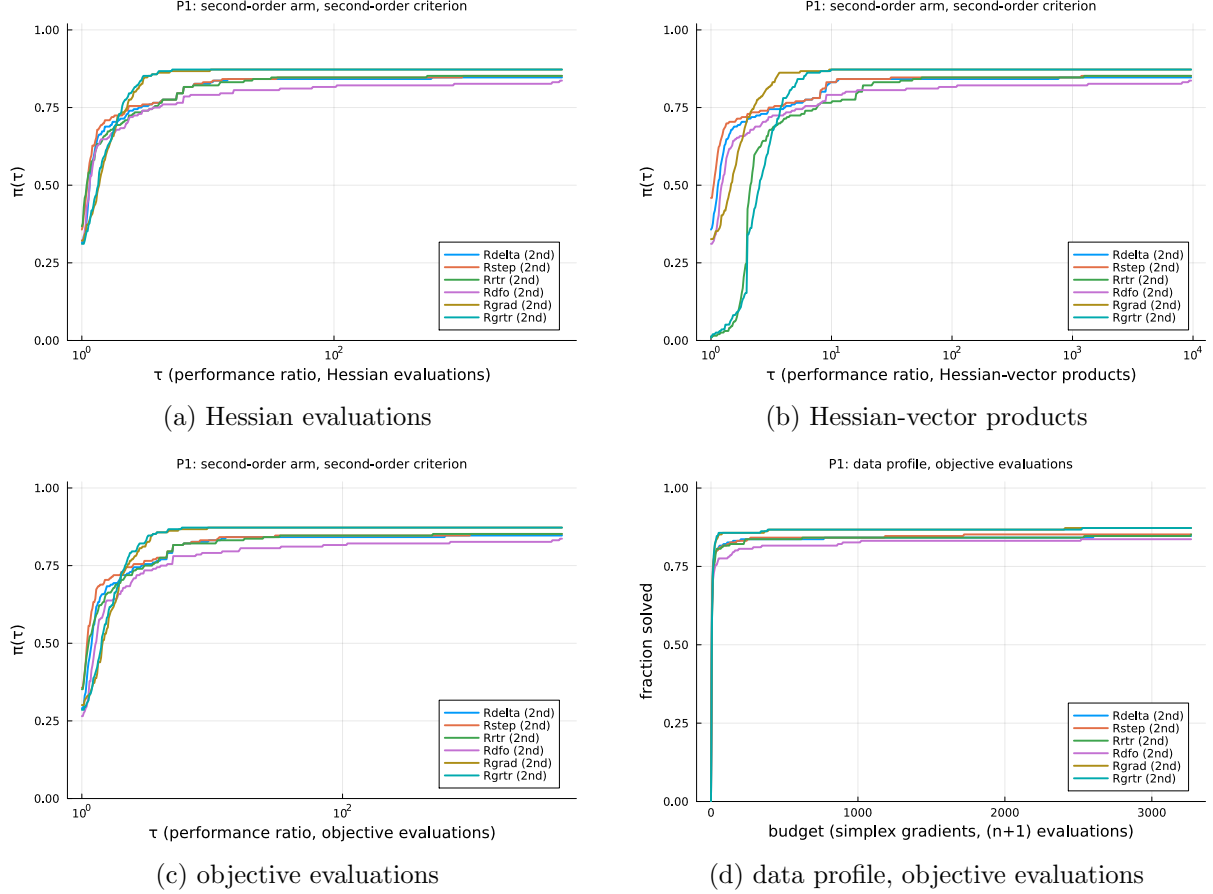


Figure 2: P1. Six second-order configurations on the second-order criterion. R-delta, R-step and R-RTR do not read a criticality measure, so for them these lines are the unmodified rule under a second-order stopping test and **ExactMS**; R-DFO, R-grad and R-gRTR read one, so their lines are the rule with $\omega_k = \tau_k$. Panel b carries the retrospective penalty discussed above and should not be read as a cost ranking on its own.

8 P2: the two arms on the first-order task

Table 5: P2. Both arms, pass g, scored by (2) at the returned point, over all 190 problems. The first-order arm’s Hessian-evaluation column is omitted because it is identically zero (Section 6).

rule	met		f evaluations		Hv products		H evaluations
	1st	2nd	1st	2nd	1st	2nd	2nd
R-delta	162	167	64 814	52 744	565 608	187 818	94 276
R-step	134	167	362 834	68 854	1 426 641	248 594	117 785
R-RTR	165	168	34 328	55 570	418 553	213 961	96 605
R-DFO	157	165	87 814	118 231	595 800	260 486	216 592
R-grad	156	171	53 651	52 957	473 479	196 974	82 609
R-gRTR	156	170	56 681	41 842	497 704	241 731	63 242

The expected reading of P2 was that the second-order arm would be the expensive one. It is not the reading the table supports.

Robustness goes to the second-order arm in every row.

By 3 to 33 problems. R-step is the extreme: 134 against 167. The second-order configuration reaches *more* first-order points than the first-order one.

So do Hessian-vector products, in every row,

by 1.96 to 5.74 times.

Objective and gradient evaluations are genuinely mixed.

R-RTR is $1.62\times$ cheaper in the first-order arm; R-step is $5.27\times$ cheaper in the second-order arm; R-grad is a wash.

Stalling is the mechanism.

Over the 2 280 runs of each arm, pass g: the first-order arm stalled 104 times and hit the iteration cap 36, the second-order arm 51 and 15.

The qualification is unavoidable and belongs in every sentence quoting these numbers. Table 2 lists three differences between the arms, and all three are active in P2. A statement of the form “R-delta saves 377 790 Hessian-vector products and pays 94 276 Hessian evaluations for them” is true, and is a statement about **ExactMS** against **SteihaugCG**. It is not a measurement of what second-order stopping costs. Section 13 gives the cell that would separate them.

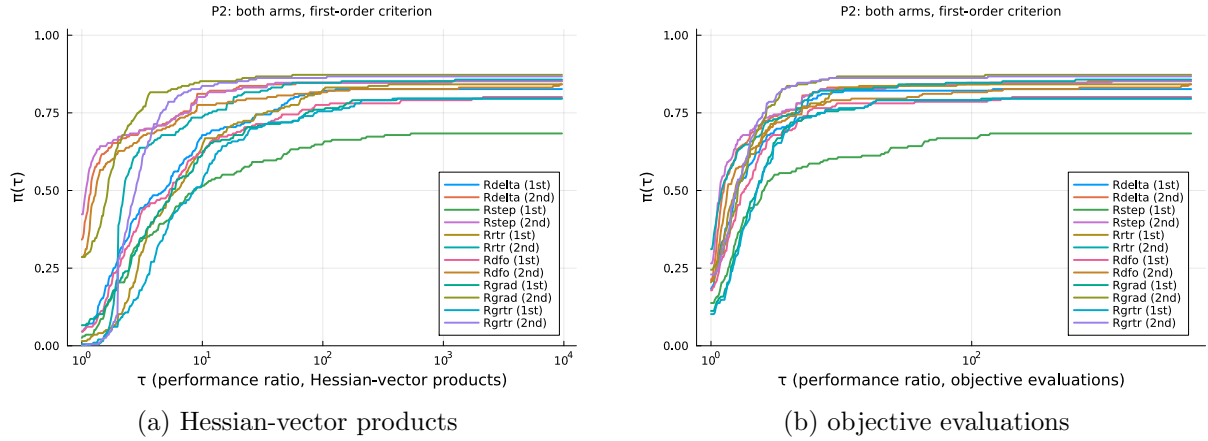


Figure 3: P2. Both arms on the first-order criterion, twelve lines. The (1st) lines use **SteihaugCG** and the (2nd) lines **ExactMS** at $n \leq 200$; the arms also differ in the stopping test and, for R-DFO, R-grad and R-gRTR, in ω_k . The profile on Hessian evaluations is not shown here and is withdrawn in Section 11.

9 P3 measured nothing

All six first-order columns of P3 read 0 out of 179. That is not a finding about the first-order arm. It is what the scoring function returns when the field it tests is absent.

The chain is short and each link is in the package. **TRSolver** computes a curvature estimate only when `want_curv` holds, and `want_curv` is `needs_curvature(rule)  $\vee$  tol_H > 0  $\vee$  needs_eigenvector(subsolver)`. In the first-order arm the rule is unwrapped so the first disjunct is false, `tol_H = -1` so the second is false, and **SteihaugCG** requests no eigenvector so the third is false. `attach_trace!` then omits `:lambda_min_trajectory` altogether, by design, so that its absence is unambiguous rather than a column of NaN. The experiment reads the missing trajectory back as NaN, and the scoring predicate opens with `isfinite`. The criterion is therefore false for every first-order run before (3) is evaluated at all.

Table 6: P3, and why it is empty. The left pair is the criterion; the right pair counts the runs that carried a finite $\lambda_{\min}$ for it to be tested on. Second-order arm, pass h, $n \leq 200$.

rule	met, of 179		runs with a finite $\lambda_{\min}$	
	1st	2nd	1st	2nd
R-delta	0	158	0	179
R-step	0	158	0	179
R-RTR	0	159	0	179
R-DFO	0	155	0	179
R-grad	0	161	0	179
R-gRTR	0	161	0	179

The second-order columns are sound — 155 to 161 of 179 — but they are P1 restricted to $n \leq 200$ and carry no information P1 does not already have. Everything specific to P3 is lost, and none of it is recoverable from the archive: the first-order arm’s returned points are stored, but the runs cannot be re-scored without evaluating $\lambda_{\min}(\nabla^2 f)$ there, which is a computation and not a lookup.

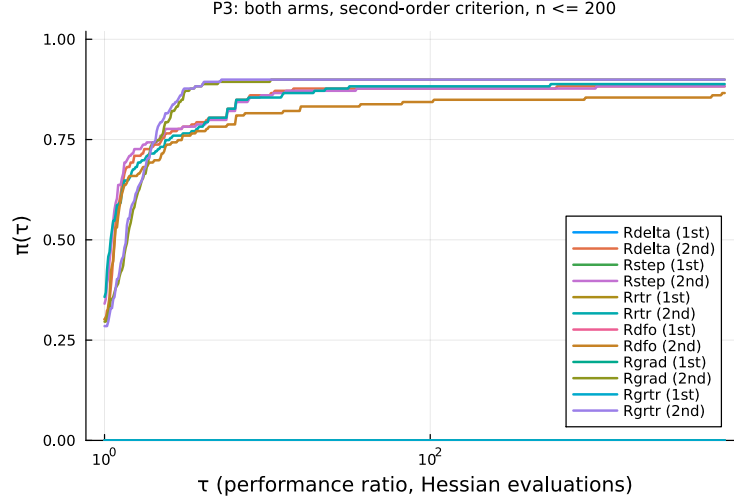


Figure 4: P3 as drawn. The six (1st) lines are flat at 0 across the whole axis. Read as a profile the figure says that the first-order arm never reaches a second-order point; read correctly it says that the campaign never asked.

10 What the archive still answers

The first-order arm’s curvature is gone. Three measurements that survive answer most of what P3 was for, and the first of them changes how the whole campaign should be read.

10.1 The first-order point was already second-order

Pass g of the *second-order* arm halts at the first iterate with $\|g_k\| \leq \varepsilon_g$ and has computed $\lambda_{\min}$ there, because that arm keeps $\text{tol.H} > 0$ in both passes. That is exactly the measurement P3 wanted, taken on the arm that recorded it.

Table 7: Curvature at the first-order point. Second-order arm, pass g, $n \leq 200$, so $\lambda_{\min}$ is a dense eigendecomposition of $\nabla^2 f(x)$ and exact.

rule	reached	$\lambda_{\min} \geq -\varepsilon_H$	$\lambda_{\min} < -\varepsilon_H$	share	median $\lambda_{\min}$ when negative
R-delta	159	155	4	2.5%	$-3.77 \cdot 10^{-5}$
R-step	158	155	3	1.9%	$-4.78 \cdot 10^{-6}$
R-RTR	159	155	4	2.5%	$-3.14 \cdot 10^{-5}$
R-DFO	156	151	5	3.2%	$-2.94 \cdot 10^{-5}$
R-grad	161	157	4	2.5%	$-1.26 \cdot 10^{-5}$
R-gRTR	160	157	3	1.9%	$-2.40 \cdot 10^{-6}$

On 96.8–98.1% of problems the point at which the gradient test first passed *already* satisfied $\lambda_{\min} \geq -\varepsilon_H$. Between 3 and 5 problems per rule had genuine negative curvature there, and the magnitudes are 10^{-5} to 10^{-6} , barely past ε_H . On this problem set at these tolerances the second-order task is very nearly the first-order task.

10.2 Certifying the curvature cost nothing

Table 8: Cost of pass h relative to pass g within the second-order arm, median over the problems on which both met their criterion, $n \leq 200$.

rule	problems	f	g	iterations	H
R-delta	158	1.000	1.000	1.000	1.000
R-step	158	1.000	1.000	1.000	1.000
R-RTR	159	1.000	1.000	1.000	1.000
R-DFO	155	1.000	1.000	1.000	1.000
R-grad	161	1.000	1.000	1.000	1.000
R-gRTR	160	1.000	1.000	1.000	1.000

Every entry of Table 8 is 1.000. On the median problem the second-order arm took no iteration beyond the one that satisfied the gradient test, which is Section 10.1 restated as a cost. The arm’s overhead is therefore not a search for curvature. It is the per-iteration price of forming $\nabla^2 f$ and eigendecomposing it, paid on every iteration of every problem, to certify something that was already true.

10.3 Where the two arms landed

Table 9: Final objective, first-order arm against second-order arm, pass h, $n \leq 200$, on problems where the first arm met (2) and the second met (3). “Same point” is a relative gap below 10^{-8} .

rule	compared	2nd lower	1st lower	same point	median gap
R-delta	147	22	4	121	$5.30 \cdot 10^{-6}$
R-step	121	18	4	99	$3.49 \cdot 10^{-6}$
R-RTR	150	21	5	124	$5.01 \cdot 10^{-6}$
R-DFO	142	16	11	115	$2.45 \cdot 10^{-7}$
R-grad	142	19	9	114	$9.20 \cdot 10^{-7}$
R-gRTR	143	19	11	113	$6.68 \cdot 10^{-8}$

The direction leans the second-order way in every row — 16 to 22 problems against 4 to 11 — but the majority is “same point” and the tail runs both ways. Only 48 of 1074 (problem, rule) pairs differ by more than 10^{-4} in relative objective.

Table 10: The pairs where the arms genuinely disagreed: the sixteen largest relative gaps, of 48 exceeding 10^{-4} . One row per (problem, rule) pair, because on MGH17SLS and BIGGS6 the direction differs between rules and a per-problem row would state neither finding.

problem	n	rule	f , 1st arm	f , 2nd arm	
POWERSUM	4	R-step	$3.74 \cdot 10^1$	$1.94 \cdot 10^{-27}$	2nd lower
POWERSUM	4	R-RTR	$3.74 \cdot 10^1$	$1.01 \cdot 10^{-21}$	2nd lower
POWERSUM	4	R-DFO	$3.74 \cdot 10^1$	$2.06 \cdot 10^{-17}$	2nd lower
POWERSUM	4	R-delta	$3.74 \cdot 10^1$	$2.96 \cdot 10^{-16}$	2nd lower
RAT43LS	4	R-gRTR	$8.79 \cdot 10^3$	$1.08 \cdot 10^6$	1st lower
MGH17SLS	5	R-DFO	$1.02 \cdot 10^0$	$2.45 \cdot 10^{-2}$	2nd lower
MGH17SLS	5	R-RTR	$1.02 \cdot 10^0$	$2.45 \cdot 10^{-2}$	2nd lower
MGH17SLS	5	R-delta	$1.02 \cdot 10^0$	$2.45 \cdot 10^{-2}$	2nd lower
MGH17SLS	5	R-step	$1.02 \cdot 10^0$	$2.45 \cdot 10^{-2}$	2nd lower
LOGHAIRY	2	R-RTR	$6.53 \cdot 10^0$	$1.82 \cdot 10^{-1}$	2nd lower
BOXBODLS	2	R-gRTR	$1.17 \cdot 10^3$	$9.77 \cdot 10^3$	1st lower
BOXBODLS	2	R-grad	$1.17 \cdot 10^3$	$9.77 \cdot 10^3$	1st lower
COATING	134	R-DFO	$5.05 \cdot 10^{-1}$	$3.13 \cdot 10^0$	1st lower
LSC2LS	3	R-grad	$1.34 \cdot 10^1$	$4.04 \cdot 10^0$	2nd lower
LSC2LS	3	R-gRTR	$1.34 \cdot 10^1$	$4.04 \cdot 10^0$	2nd lower
BIGGS6	6	R-step	$5.66 \cdot 10^{-3}$	$2.43 \cdot 10^{-1}$	1st lower

POWERSUM is the case the second-order theory is written for: the first-order arm stops at $f \approx 37.4$ and the second-order arm reaches machine zero, on four of the six rules. MGH17SLS and LOGHAIRY are the same story at smaller amplitude. But RAT43LS, BOXBODLS, COATING and BIGGS6 run the other way. This is not a contradiction: (3) constrains the curvature at the point reached and says nothing about which basin it lies in. It is, however, the reason the payoff cannot be reported as an average, and must be reported as this asymmetry.

11 Defects

Four, in descending order of what they cost the conclusions.

D1. P3’s first-order columns are an instrumentation artefact.

Section 9. 0 of 2 280 first-order runs carry a finite $\lambda_{\min}$, against 2 276 of 2 280 in the second-order arm. The argument of Section 2.3 anticipated this hazard for the wrong arm: it reasons that `tol_H` must not be switched off in pass `g` because that would switch off the cost the pass exists to measure, and then switches it off in the first-order arm in both passes, which switches off the measurement P3 exists to make. **P3 should not be reported** — not as a result, not as a null result, not with a caveat.

D2. The P2 profile on Hessian evaluations is degenerate.

The first-order arm spends none, and `sop_matrix` guards a zero cost to 1 so that the per-problem normalisation does not divide by zero. Every first-order line therefore wins outright at $\tau = 1$ and is flat thereafter, and each second-order line is not a ratio but its own raw Hessian-evaluation count on a logarithmic axis. `exp17_P2_h_evals.pdf` is withdrawn. The comparison it was meant to carry is in Table 5, where 0 against a positive number is stated plainly.

D3. The two arms differ in three ways at once.

Table 2 lists them and Section 2.1 defends the subsolver switch on its merits — `SteihaugCG` does not deliver (4), so the second-order arm genuinely requires a different solver. The consequence for the write-up stands regardless: P2 compares configurations, not second-order-ness. The same confound means this campaign *cannot* answer whether $\omega_k = \tau_k$ helps, because no pair of columns differs in the wrapper alone.

D4. 6 problems that never opened are counted as solver failures.

Each produced 24 placeholder records with $n = 0$. Three knock-on errors follow. Hazard H4 reports 148 runs raised; the true figure is 4, all on `EXTROSNB` at $n = 1000$, and the other 144 never reached the solver. Every status table’s exception column is inflated by 6 and every denominator reads 196 where 190 problems have data. And `exp17_problems.txt` reports 179 problems at $n \leq 200$ and 17 above, where only 11 are above and the remaining 6 are the placeholders — which is also why Section 4 anticipated 185 small problems and the run delivered 179. Worth reproducing before it is treated as a stable CUTEst failure: `ARGLINA` and `ARGLINC` both opened successfully during the checks of Section 3, minutes earlier, and `ARGLINB` opened in the campaign itself.

12 What this run supports

P1, without qualification.

Six second-order configurations on the second-order task separate by at most five percentage points in robustness and by nearly a factor of three in cost. `R-gRTR` is jointly the most robust and the cheapest in objective, gradient and Hessian evaluations; `R-DFO` is last on both. The *Hv* column of the two retrospective rules is the retrospective ratio’s extra product and carries no further meaning.

P2, with the confound named in the sentence.

The second-order configuration is more robust than its first-order counterpart on the *first-order* task, for every rule, by 3 to 33 problems, and uses 1.96 to 5.74 times fewer Hessian-vector products, paying for both with a dense Hessian and an eigendecomposition per iteration. Stated as a comparison of configurations this is a usable result. Stated as “the price of the second-order machinery” it is not.

P3, not at all.

Its first-order columns record the absence of a measurement, not the outcome of one, and there is no reading of them that is about the algorithms.

The finding that reframes the campaign.

On CUTEst at $\varepsilon_g = 10^{-5}$ and $\varepsilon_H = 10^{-6}$, the first-order point was already second-order on 96.8–98.1% of problems, and the second-order arm spent zero extra iterations reaching it on the median problem. What the machinery buys here is a *certificate*, not a better point, and it buys it at full price on all 190 problems in order to matter on about five. Where it matters it matters completely — on `POWERSUM` the objective falls from 37.4 to machine zero. Where it goes the other way it also goes completely. That asymmetry, and not an average, is the honest statement of the payoff.

Remark 1. These are average-case results on one problem set at one pair of tolerances. Nothing here bears on the sharpness of a worst-case complexity bound, and Section 10.1 suggests the problem set is the binding constraint on what the second-order half of the study can observe at all.

13 Changes for the next run

1. **Score the first-order arm’s curvature after the fact.** The arm must reach its point using first-order machinery, then have $\lambda_{\min}$ evaluated once at the returned point purely for scoring. One call to `lambda_min_estimate(ExactHessian(), nlp, st.solution)` in `sop_solve` after `tr_solve` returns, charged to no counter, is the whole fix: one extra `hess` per run, and the arm’s search behaviour untouched. This repairs D1 and makes P3 the profile it was designed to be.
2. **Withdraw the Hessian-evaluation axis from P2, or change its currency.** A metric one arm cannot spend is not a metric two arms can be profiled on. Either drop it from `SOP_METRICS` for P2, or replace both Hessian axes by a single $Hv + n \cdot H$ cost so the arms spend in the same units. This repairs D2.
3. **Separate “could not open” from “raised”.** `sop_failed_row` should carry a distinct status, and those problems should leave every denominator rather than count as failures. This repairs D4, hazard H4, the exception columns and the $n > 200$ tally at once.
4. **Add the third arm.** A plain criticality-anchored rule with `ExactMS` and `tol_H =  $\varepsilon_H$` differs from the second-order arm in the `SecondOrder` wrapper alone. Three arms rather than two turn P1’s ranking into an attribution, and the question it answers — whether $\omega_k = \tau_k$ earns its place — is the one the wrapper exists to raise. This repairs D3.
5. **Promote Table 7.** It is the most decision-relevant measurement in the campaign, it is currently in no shipped table, and it sets the context in which every other result here has to be read. If the second-order question is to be pressed further, the problem set should be chosen so that this table is not 97% trivial.